

Optimizing Small Language Models (SLMs) for Syntax-Agnostic, Resource-Efficient C/C++ Vulnerability Detection

Jonathan Shanmuganantham
University of Waterloo

Abstract

On-device Small Language Models (1B to 12B parameters) are a lightweight, privacy-preserving alternative to cloud-hosted frontier models for static code analysis inside continuous-integration pipelines. I fine-tune and evaluate five on-device models against frozen gpt-5.6-luna on source and on compiler-derived LLVM IR, across a synthetic benchmark (Synthset, 957 oracle-verified programs) and a real-world benchmark (Realset, 2,995 minimal-diff CVE pairs proven disjoint from training). Training uses reasoning distillation and supervised fine-tuning with a preference-optimization stage on top, all on AWS g6e instances with one NVIDIA L40S (48 GB) GPU per job, at a total compute cost of \$404. What I find splits by representation and by metric. On clean synthetic source a 9B model reaches parity, with balanced accuracy 0.626 against 0.629 at one third the false-positive rate and higher weakness naming (0.821 against 0.765). On real code a discrimination wall holds every on-device model at chance (0.50 to 0.51) while the frontier reaches only 0.545. The small models still lead on pair-consistency (0.226 against 0.156) and family-level naming (0.44 against 0.36), and a 2.6B model names real weaknesses about as accurately as the frontier. Compiling to LLVM IR does not help the small model. On the same programs it detects and names better on source, and IR aids only the frontier, cutting its false-positive rate while lowering its recall of real bugs. Reasoning distillation is the lever behind the real-code gains, lifting pair-consistency from 0.099 to 0.209 where format-only imitation reaches 0.141, and preference optimization shifts the operating point without crossing the wall. No model is a standalone real-code detector, so the practical deployment strategy is a detect-then-name cascade with a reviewer in the loop. The generator and its oracles, the training and evaluation code, the packaged datasets, and the checkpoint manifest are released at <https://github.com/jshan9078/cwe-model-training>.

1 Introduction and Motivation

Manual code review cannot scale to enterprise repository throughput. Automated static application security testing tools such as SonarQube, CodeQL, and Semgrep detect flaws deterministically but suffer from false positive rates exceeding 68%, imposing severe audit overhead. Proprietary cloud-hosted Large Language Models achieve higher recall but introduce data privacy risks, network latency, and continuous API costs.

On-device Small Language Models (1B to 12B parameters) provide a secure, cost-effective alternative for local continuous integration environments. However, current small models trained directly on raw C/C++ source code exhibit lexical

fragility. Subtle formatting shifts, identifier renamings, or loop refactorings frequently cause classification failure. Prior work establishes that security reasoning recruits sparse subnetworks inside model weights, indicating that targeted fine-tuning on structured representations and distilled reasoning traces can overcome surface sensitivity.

This project addresses these challenges through a three-stage optimization framework.

1. **Representation Abstraction** Compiling raw C/C++ source code into LLVM Intermediate Representation to eliminate surface-level lexical noise and normalize variables into Static Single Assignment registers.
2. **Off-Policy Reasoning Distillation and SFT Fine-tuning** Training small models on verifier-gated reasoning traces distilled from frontier models alongside real-world CVE pairs.
3. **Direct Preference Optimization** Aligning model decision boundaries using preference pairs constructed from deterministic sanitizer verifiers on paired vulnerability variants.

1.1 Research Questions

I address three primary research questions in this study.

- **RQ1** How far do 1 to 12B on-device models trail gpt-5.6-luna (high) on CWE classification, measured by balanced accuracy, naming, and pair-consistency, and how does the gap scale with model size and architecture?
- **RQ2** Does compiler-derived LLVM IR help or hurt a small model, relative to source, for the same task? A control that runs gpt-5.6-luna (high) on IR separates a small-model benefit from IR being easier for every model.
- **RQ3** How much does reasoning-distillation supervised fine-tuning, and preference-based reinforcement learning on top of it, close the gap over the untuned baselines?

1.2 Related Work

Recent literature highlights both the potential and limitations of language models in software vulnerability detection across 13 core publications grouped into five thematic domains.

1.2.1 SLM and LLM Vulnerability Benchmarks

Müller-Uri and Pykälistö [8] benchmarked local models (CodeLlama-7B, Qwen3-1.7B, Mistral-7B, DeepSeek-Coder-6.7B, Llama-3.2-3B) against cloud models (Claude Opus 4.7 and ChatGPT-5.4) on PrimeVul. Their findings established that SFT primarily improves output format adherence rather than underlying domain reasoning, while in-context few-shot examples yield negligible benefits for fine-tuned models.

Lin and Mohaisen [9] evaluated 14 models on Juliet and DiverseVul. They demonstrated that parameter scaling does not guarantee detection improvement, quantization sensitivity is model-dependent, and small models struggle with multi-class weakness enumeration.

Henareh [10] fine-tuned Llama-3.1-8B on cybersecurity instruction datasets, showing that general security fine-tuning without code-specific data yields minimal gains (1.4% to 1.6%) and degrades reasoning, underscoring the necessity of code-representation-specific tuning.

1.2.2 Circuit Analysis and Mechanistic Interpretability

Al Atiq et al. [7] conducted circuit-level causal analysis on Gemma-2-2B during vulnerability detection tasks. They discovered that vulnerability reasoning is highly sparse, where only 2 out of 26 layers participate in classification, and zeroing out top selective neurons reduces detection performance by over 50%.

1.2.3 Reinforcement Learning for Vulnerability Detection

Lu et al. [11] proposed DAGVUL, applying Reinforcement Learning with Verifiable Rewards via GRPO to Qwen3-8B. By rewarding logical dependency reasoning, DAGVUL achieved 70.97% zero-shot accuracy (+17% over base model) and 76.11% CoT accuracy.

Rondanini et al. [12] demonstrated LoRA-based continuous learning on edge devices for malware detection, showing adapters with under 1% parameter overhead improved accuracy by 20% to 25%.

Kozachok and Kozachok [13] reviewed small language models for domain-specific language generation, establishing a complex taxonomy that maps PEFT to structured representations.

1.2.4 Code Representations and SAST Integration

Gneciak and Szandala [5] compared GPT-4.1, Mistral Large, and DeepSeek V3 against SonarQube, CodeQL, and Snyk-Code. LLMs achieved higher F1-scores but suffered higher false positive rates and tokenization ambiguities.

Hu et al. [14] introduced QLPro, integrating LLMs with CodeQL via a three-role collaborative loop, achieving 90.9% rule compilability and detecting 66.1% of vulnerabilities.

Jin [15] surveyed five LLM adaptation paradigms, observing that decoder-only models exhibit high precision but lower recall.

Xiong and Zhang [16] evaluated LLM agents for false positive filtering, reducing false positive rates from above 98% down to 6.3%.

1.2.5 Statement-Level and Paired Benchmarks

Ahmed et al. [17] introduced SECVULEVAL, a statement-level benchmark of 25,440 functions across 5,867 CVEs, showing open-source models achieved only 7.14% to 13.57% statement F1.

Finally, Li et al. [6] evaluated pre-trained code models on PrimeVul, proving that performance drops significantly under control-flow shifts or inline comments.

2 Summary about Data Section

The data infrastructure delivers a shortcut-resistant, machine-verifiable C/C++ weakness corpus alongside real-world CVE evaluation pairs. The synthetic universe provides 14,166 samples across 24 weakness classes where surface syntax is statistically independent of labels. The real-world universe provides 2,995 minimal-diff CVE pairs from BigVul and 2,481 pairs from PrimeVul. Every synthetic sample is verified by dynamic runtime oracles (AddressSanitizer, UndefinedBehaviorSanitizer, MemorySanitizer, ThreadSanitizer, and exploit harnesses), ensuring zero ground-truth ambiguity. Off-policy distillation extracts 638 verifier-gated reasoning traces for source (637 for LLVM IR) from frontier models, providing distilled reasoning for supervised fine-tuning.

2.1 External Datasets Used for Real World Data

To evaluate external validity on human-written production code, I ingest three primary external datasets.

- **BigVul (Fan et al.) [1]** Mined from real-world C/C++ GitHub repositories, providing CVE-fixing commit functions. I construct **Realset**, a clean, provably train-disjoint evaluation set of 2,995 minimal-diff pairs (5,990 functions total) with median diff length of 9% of lines.
- **PrimeVul (Ding et al.) [2]** Reconstructed CVE dataset used for real-code supervised fine-tuning (**REALSFT**) comprising 3,128 clean real-world CVE examples hash-deduped against all evaluation sets.
- **SARD Juliet C/C++ Suite v1.3 [3]** SARD benchmark suite used strictly as training data augmentation (7,980 samples) and never as an evaluation set.

2.2 Synthetic Data Created

My synthetic generator composes a defect mechanism (104 available shapes) with an authored context (50 program hosts) under 16 independent surface-variation dimensions (identifiers, buffer sizes, allocators, control flow structures). Samples are generated as twin pairs (vulnerable and secure). A coin flip releases exactly one twin per sample, tying bag-of-tokens classifiers to chance.

2.2.1 Generator and Oracle Verification

Every synthetic label is verified by a per-class dynamic runtime oracle before admission. Spatial and temporal memory errors require matching AddressSanitizer report signatures. Integer errors require UndefinedBehaviorSanitizer triggers. Uninitialized reads require MemorySanitizer reports. Concurrency flaws require ThreadSanitizer. Injection flaws require exploit harness verification.

Table 1 presents Python implementation details of my automated AddressSanitizer and UndefinedBehaviorSanitizer verification parser module.

2.3 Benchmarks Created for Testing

I evaluate all models on two disjoint primary benchmarks.

- **Synthset (Synthetic Discrimination Benchmark)** 957 oracle-verified C/C++ programs across 24 weakness classes (balanced 20 per class). Surface features sit at chance under statistical probes, ensuring scores reflect security reasoning

Table 1: Dynamic Oracle Verification Parser Code

Oracle Sanitizer Parser Implementation (Python)
<pre>def parse_asan_report(output: str) -> dict: if "heap-buffer-overflow" in output: kind = "CWE-122" elif "stack-buffer-overflow" in output: kind = "CWE-121" elif "heap-use-after-free" in output: kind = "CWE-416" elif "attempting double-free" in output: kind = "CWE-415" else: kind = "UNKNOWN" is_w = "WRITE on unknown address" in output is_r = "READ on unknown address" in output return {"cwe": kind, "write": is_w, "read": is_r, "valid": kind != "UNKNOWN"} def verify_ubsan_integer(output: str) -> str: if "unsigned integer overflow" in output: return "CWE-190" if "shift out of bounds" in output: return "CWE-369" return "SECURE"</pre>

rather than lexical pattern matching.

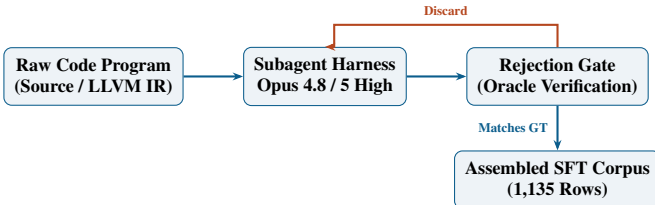
- **Realset (Real-World Transfer Benchmark)** 2,995 minimal-diff real-CVE pairs (5,990 functions) from BigVul, provably disjoint from all training data. Measures whether models can resolve a real vulnerability from its one-commit fix.

For the direct source-versus-IR comparison I additionally define the **Deployable-733** subset, the 733 of the 957 Synthset programs whose compiled LLVM IR fits the 16,384-token training window. The remaining 224 programs exceed that window and are dropped from IR training (Section 4.3), so scoring both representations on this identical 733-program set removes the confound that IR must discard its longest programs. Neither representation is then advantaged by sample selection, which is the fair basis for the head-to-head in Section 4.4.

2.4 Off-Policy Distillation Procedure

Training traces are distilled off-policy from frontier teacher models (Claude Opus 4.8 and Opus 5 high reasoning effort). This is not knowledge distillation in the classic sense. A hosted teacher API returns no internal activations, output logits, or hidden reasoning tokens, so I cannot match its output distribution or transfer its latent thinking. I instead instruct the teacher to write its analysis as ordinary output text and treat that written reasoning as the supervision target. Each teacher is executed inside an isolated subagent in the Claude Code harness. The subagent receives a single program without labels or cross-sample context.

Figure 1 illustrates the subagent distillation pipeline.

**Figure 1:** Off-Policy Subagent Distillation Pipeline

The exact distillation system prompt and prompt format are presented in Table 2.

2.5 Dataset Construction and Counts

Table 3 details sample counts across splits and assembled SFT components.

Table 2: Exact Distillation Prompt and Sample JSON Output

Distillation System Prompt

You are a senior security engineer reviewing production C/C++ (or LLVM IR) before it ships. Follow the values that bound each access, allocation, release or interpreter call. State which specific operation is or is not safe and why, quoting concrete symbols. SIBLING DISCRIMINATION (mandatory) when you choose a CWE you MUST name the single closest sibling from the list and cite the one concrete fact that rules it out. Choose the verdict ONLY from the closed CWE list, or SECURE. Answer ONLY as JSON with two keys {"think": "<120-300 words reasoning>", "verdict": "<CWE-XXX or SECURE>"}. {"think": "The function allocates buffer iItemRef using new short[FIELDPREF+CHUNKREF]. At line 14, iItemRef is set to NULL before calling delete[] iItemRef. Calling delete[] on a NULL pointer is a no-op, leaving the previously allocated memory orphaned on the heap without deallocation. This constitutes an unreleased resource flaw. Sibling 415 (double free) is ruled out because free/delete is never called twice on the valid pointer. Sibling 416 (use-after-free) is ruled out because the memory is never accessed post-nullification. Therefore this is a heap memory leak.", "verdict": "CWE-401"}

Sample Teacher Distillation Output

{"think": "The function allocates buffer iItemRef using new short[FIELDPREF+CHUNKREF]. At line 14, iItemRef is set to NULL before calling delete[] iItemRef. Calling delete[] on a NULL pointer is a no-op, leaving the previously allocated memory orphaned on the heap without deallocation. This constitutes an unreleased resource flaw. Sibling 415 (double free) is ruled out because free/delete is never called twice on the valid pointer. Sibling 416 (use-after-free) is ruled out because the memory is never accessed post-nullification. Therefore this is a heap memory leak.", "verdict": "CWE-401"}

Table 3: Dataset Partition and SFT Component Counts

Dataset Partition	Source Samples	LLVM IR Samples
Custom Synthetic Train	7,889	7,889
Custom Synthetic Validation	2,157	2,157
Custom Synthetic Test (Pool)	4,120	4,120
Synthset Eval (Headline Draw)	957	957
Deployable-733 (IR fits)	733	733
Realset Eval (BigVul Disjoint)	5,990	N/A
PrimeVul CVE Train (REALSFT)	3,128	N/A
Juliet SARD Training Aug.	7,980	7,980
Assembled SFT Mixture		
Custom Distillation Traces	638	637
Juliet Augmentation	230	236
CWE Knowledge Cards	267	267
LLVM Knowledge Cards	N/A	230
Total Assembled SFT	1,135	1,370

3 Model Summary and Initial Scores

I evaluate five target architectures spanning active parameter scale, architectural sparsity, and reasoning capacity, alongside gpt-5.6-luna as the frontier ceiling. Table 4 presents model registry properties.

3.1 Metrics Used

To rigorously evaluate model performance across binary detection, multi-class naming, and real-world minimal-diff pair discrimination, I define ten primary quantitative metrics. Table 5 provides the complete registry of metrics, column header abbreviations, and exact formulas.

Binary detection accuracy is evaluated using balanced accuracy (A_{bal}), which accounts for class imbalance by averaging true positive rate (R) and true negative rate (S).

$$A_{bal} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \quad (1)$$

For paired real-world CVE functions (v, s) where v is vulnerable and s is the patched secure twin, pair-consistency (pair) measures the proportion of pairs where the model correctly flags v and clears s . Pair-consistency decomposes into pair split rate (split) and correct-flip rate (cflip), satisfying pair = split $\times$ cflip. Within-pair decision correlation (ϕ) is calculated as follows.

$$\phi = \frac{|P|n_{11} - n_1m_1}{\sqrt{n_1(|P| - n_1)m_1(|P| - m_1)}} \quad (2)$$

Table 4: Model Registry and Properties

Model	Arch.	Params	Active	Context
LFM2.5-8B-A1B	MoE	8.0B	1.5B	128k
LFM2.5-2.6B	Hybrid	2.6B	2.6B	128k
Qwen-3.5-9B	Dense	9.0B	9.0B	262k
Gemma-4-12B-it	Dense	12.0B	12.0B	262k
Coder-7B	Dense	7.0B	7.0B	128k
gpt-5.6-luna	Frontier	>1.0T	n/d	1M

Table 5: Complete Metrics Registry and Mathematical Definitions

Header	Full Metric Name	Mathematical Formula / Definition
A_{bal}	Balanced Accuracy	$\frac{1}{2}(R + S)$ where $R = \frac{TP}{TP+FN}$, $S = \frac{TN}{TN+FP}$
catch	Recall (TPR)	$R = \frac{TP}{TP+FN}$
FPR	False Positive Rate	$FPR = \frac{FP}{FP+TN} = 1 - S$
name	CWE Naming (Graded)	$d(p, g)$ over flagged true bugs, Eq. (2)
+fam	Family Naming	$d_{\text{fam}}(p, g) = \max_{q \in \{p\} \cup \text{anc}_{\leq 1}(p)} d(q, g)$
pair	Pair-Consistency (PCR)	$\frac{1}{ P } \sum_{(v,s) \in P} \mathbf{1}[f(v)=1 \wedge f(s)=0]$
split	Pair Split Rate	$\frac{1}{ P } \sum_{(v,s) \in P} \mathbf{1}[f(v) \neq f(s)]$
cflip	Correct-Flip Rate	pair/split
ϕ	Within-Pair Correlation	Eq. (3)
tok	Mean Output Length	Average generated output tokens per program

where $n_1 = \sum f(v)$, $m_1 = \sum f(s)$, and $n_{11} = \sum f(v)f(s)$.

Hierarchical weakness naming ($d(p, g)$) assigns partial credit based on MITRE CWE tree distance [4] between predicted leaf p and gold standard g .

$$d(p, g) = \begin{cases} 1.0 & p = g \\ 0.5 & p \in \text{anc}(g) \vee g \in \text{anc}(p) \\ 0.25 & \text{parent}(p) = \text{parent}(g) \\ 0.0 & \text{otherwise} \end{cases} \quad (3)$$

3.2 Initial Benchmark Scores of Selected Models

Table 6 presents zero-shot untuned baseline evaluations on Synthset (957 programs) across Source and LLVM Intermediate Representation. Throughout the results tables, **bold** marks the best on-device value in each column, and gpt-5.6-luna is shown as the reference ceiling rather than a competitor.

Table 6: Untuned Baseline Benchmark Scores (Synthset 957). Naming is exact CWE. Coder-7B was run untuned on source only, where the terse verdict format gags a reasoning-oriented code model into a chance-level under-flagger.

Model	Rep	binAcc	catch	FPR	name	ms/ex
LFM2.5-8B-A1B	Source	0.514	0.354	0.325	0.037	719
LFM2.5-8B-A1B	LLVM	0.510	0.183	0.164	0.011	1074
LFM2.5-2.6B	Source	0.502	0.542	0.539	0.035	548
LFM2.5-2.6B	LLVM	0.510	0.273	0.254	0.076	960
Qwen-3.5-9B	Source	0.643	0.542	0.256	0.504	6762
Qwen-3.5-9B	LLVM	0.525	0.481	0.432	0.160	14476
Gemma-4-12B-it	Source	0.528	0.794	0.740	0.116	94
Gemma-4-12B-it	LLVM	0.511	0.773	0.753	0.089	1058
Coder-7B	Source	0.500	0.127	0.128	0.328	N/A
gpt-5.6-luna	Source	0.629	0.915	0.656	0.683	N/A
gpt-5.6-luna	LLVM	0.717	0.781	0.348	0.694	N/A

Table 7 presents the exact zero-shot inference prompt format used during baseline evaluation runs.

Table 7: Zero-Shot Inference Prompt Format

Zero-Shot Inference Prompt
You are a vulnerability classification system. Analyze the {rep.label} and respond with exactly one XML verdict tag. If the code is vulnerable, output <verdict>CWE-XXX</verdict>. If the code is safe, output <verdict>SECURE</verdict>. Do not output anything else outside the tag. Code ({rep.label}) {code}

4 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning instantiates structural security output priors and installs weakness naming capabilities. Across all architectures, SFT eliminates parse errors (reducing unparseable output from up to roughly 6% (58 of 957) down to under 0.1%). Parameter rank sweeps demonstrate that adapter capacity drives per-flag naming skill, while training corpus composition dictates false positive calibration.

4.1 Hardware Used for SFT on AWS and Total Cost

Supervised fine-tuning was executed on self-hosted AWS cloud infrastructure. Compute resources comprised four self-terminating AWS g6e.xlarge nodes (NVIDIA L40S 48GB VRAM) and one on-demand p4d.24xlarge node (8x NVIDIA A100 40GB VRAM, 96 vCPUs in us-east-1). Headless user-data execution scripts automated S3 code pull, venv setup, LoRA training, vLLM evaluation, S3 artifact upload, and node termination. Across all exploratory sweeps, rank evaluations, and final training runs, the total AWS compute cost was **\$404 USD**.

4.2 SFT for Source C/C++

I evaluate seven primary source-tuned configurations across LoRA ranks $r=4$, $r=8$, $r=16$, and $r=32$, trained over 1 to 4 epochs on different data blends.

4.2.1 Evaluated Models, Checkpoints, and Data Blends

Evaluated configurations include

- **TERSE (Qwen-3.5-9B)** LoRA r16, trained on 6,128 terse real and synthetic examples without reasoning traces over 3 epochs.
- **Coder-7B terse (Qwen2.5-Coder-7B)** LoRA r16, trained on the same 6,128-sample terse combined corpus (verdict-only, no reasoning traces) over 3 epochs. The Realset pair-consistency leader.
- **MIXED (Qwen-3.5-9B)** LoRA r16, trained on 1,161 synthetic reasoning-distillation traces combined with 3,356 terse real examples over 3 epochs (checkpoints ep1, ep2, ep3).
- **REALSFT (LFM2.5-2.6B)** LoRA r16, trained on 3,128 clean real-world PrimeVul CVE examples over 3 epochs.
- **Baseline SFT (LFM2.5-2.6B)** LoRA r4, r8, r16, r32 trained on assembled 1,135-sample synthetic mixture over 3 epochs.
- **Expanded SFT (LFM2.5-2.6B)** LoRA r16 and r32 trained on 1,641-sample FP-targeted hard-negative expanded corpus (45% secure).
- **sec55 SFT (LFM2.5-2.6B)** LoRA r16 trained on 55%-secure precision-tilted corpus.

4.2.2 Complete Results on Benchmarks

Table 8 presents complete performance results on Synthset (957 synthetic) and Realset (2,995 real CVE pairs).

Table 8: Source SFT Results Across Benchmarks. name/+fam is graded CWE-tree naming with a one-hop family fallback, scored over flagged true vulnerabilities.

Configuration	Synthset (957)			Realset (2,995)			
	binAcc	FPR	name/+fam	binAcc	FPR	pair	name/+fam
Qwen MIXED ep3 (9B)	0.626	0.224	0.821/0.847	0.514	0.499	0.209	0.142/0.205
Qwen MIXED ep2 (9B)	0.608	0.260	0.775/0.807	0.514	0.493	0.202	0.132/0.189
Qwen TERSE ep3 (9B)	0.526	0.294	0.608/0.657	0.508	0.211	0.141	0.193/0.292
Coder-7B terse (7B)	0.512	0.365	0.972/0.976	0.510	0.510	0.226	0.209/0.312
REALSFT (2.6B)	0.521	0.396	0.104/0.152	0.504	0.426	0.178	0.288/0.439
SFT r16 baseline (2.6B)	0.543	0.686	0.593/0.628	0.503	0.668	0.156	0.120/0.154
SFT r32 baseline (2.6B)	0.515	0.602	0.678/0.703	0.504	0.642	0.180	0.108/0.139
r16+exp (45% sec)	0.543	0.474	0.636/0.665	0.508	0.636	0.167	0.113/0.142
r32+exp (45% sec)	0.542	0.503	0.697/0.723	0.507	0.663	0.165	0.110/0.140
sec55 (55% sec)	0.532	0.373	0.630/0.674	0.504	0.515	0.183	0.114/0.144
gpt-5.6-luna (Frontier)	0.629	0.656	0.765/0.769	0.545	0.502	0.156	0.278/0.362

4.2.3 Analysis of Source Fine-Tuning Findings

- **Distillation traces are the primary lever for minimal-diff discrimination.** On the same Qwen-3.5-9B base, format-only SFT without reasoning traces achieves a Realset pair-consistency of 0.141, whereas adding off-policy distillation traces elevates pair-consistency to 0.209, surpassing the gpt-5.6-luna frontier baseline (0.156).
- **REALSFT dominates real-world CVE naming accuracy.** Fine-tuning the 2.6B model on real PrimeVul code yields a hierarchical naming score of 0.288 on Realset, outperforming gpt-5.6-luna (0.278) while maintaining the lowest false positive rate (0.426) among 2.6B models.
- **Recall-oriented synthetic SFTs trade false positive rate for catch.** Models trained on recall-heavy synthetic mixtures achieve high catch (0.65 to 0.68) but suffer excessive false positive rates (0.64 to 0.68), resulting in minimal-diff pair consistency collapse (0.14 to 0.16).
- **Parametric capacity ceilings limit 2.6B synthetic naming.** Sweeping LoRA rank from r4 to r32 on the 2.6B base improves per-flag naming from 0.332 up to 0.678, where it plateaus, demonstrating that 2.6B parameter capacity cannot match 9B synthetic naming levels (0.821).

4.3 SFT for LLVM IR

LLVM Intermediate Representation flattens control flow and normalizes variable identifiers into Static Single Assignment registers.

4.3.1 Training Limitations on LLVM IR

Training on LLVM IR incurs a severe token length tax. Mean token lengths expand by 6x to 9x compared to source code. Back-propagation memory consumption reaches 1.9 MB per token on L40S GPUs. To prevent Out-Of-Memory failures, I established a strict training ceiling of 16,384 tokens per step (batch size 1, gradient accumulation 32). Consequently, 61 of 1,370 training samples (4.5%) exceeding 16,384 tokens were excluded from training. At inference, 224 of 957 test programs (23.4%) exceed 16,384 tokens, requiring evaluation over the deployable-733 subset.

4.3.2 Evaluated LLVM Models and Ranks

I evaluate LoRA ranks r=4, r=8, r=16, r=32, r=64, r=128, and r=256 on LFM2.5-2.6B over 1 to 4 epochs.

4.3.3 Complete Results on LLVM Benchmarks

Table 9 presents complete LLVM IR fine-tuning results on the deployable-733 test subset.

Table 9: LLVM IR SFT Rank Sweep Results (Deployable-733 Subset). Naming is exact CWE.

Model Rank	Trainable	binAcc	catch	FPR	name
Untuned Floor	0.0M	0.513	0.325	0.299	0.083
SFT r=4	6.1M	0.499	0.566	0.569	0.154
SFT r=8	12.2M	0.509	0.642	0.626	0.232
SFT r=16	24.5M	0.502	0.631	0.629	0.284
SFT r=32	48.9M	0.516	0.729	0.701	0.365
SFT r=64	97.8M	0.501	0.694	0.695	0.496
SFT r=128	195.6M	0.475	0.599	0.651	0.648
SFT r=256	391.3M	0.509	0.596	0.580	0.636
gpt-5.6-luna	Frontier	0.707	0.778	0.365	0.658

4.3.4 Analysis of LLVM Fine-Tuning Findings

- **Rank is the primary capacity lever on LLVM IR up to rank 128.** Per-flag naming skill on LLVM IR increases monotonically with LoRA rank doublings from r4 (0.154) up to r128 (0.648), matching frontier naming performance (0.658).
- **LoRA rank scaling plateaus at rank 128.** Doubling rank from r128 to r256 (391.3M parameters, 14.4% of base) yields no additional naming gain (0.636 vs 0.648), establishing rank 128 as the architectural capacity ceiling for 2.6B LLVM IR models.
- **LLVM IR alters false positive calibration but does not elevate small model discrimination.** While frontier LLVM IR detection accuracy reaches 0.707 (+0.078 over source), small on-device models remain at chance balanced accuracy (0.475 to 0.516) across all ranks.

4.4 SFT Wrap-Up Source versus LLVM Performance

Table 10 summarizes the operational comparison between Source and LLVM IR representations.

Table 10: Source vs LLVM IR Representation Wrap-Up. Peak Naming Skill is exact CWE.

Metric / Dimension	Source C/C++	LLVM IR
Optimal LoRA Rank (2.6B)	r=16	r=128
Peak Naming Skill (2.6B)	0.684 (r32)	0.648 (r128)
Frontier Detection Accuracy	0.629	0.717
On-Device Detection Accuracy	0.543	0.509
Mean Input Prompt Length	988 tokens	22,777 tokens
Throughput Latency (ms/ex)	548 ms	960 ms
Real-World Test Availability	Yes (Realset)	No (Uncompilable)

4.4.1 Source Candidates versus the Best LLVM IR Model on Deployable-733

Table 10 aggregates the representation trade-off, and Table 11 makes it a controlled head-to-head. Every tuned source candidate and the best LLVM IR model (r128) are scored on

Table 11: Source candidates versus the best LLVM IR model on the identical Deployable-733 subset. **name** is exact CWE over flagged true vulnerabilities. On the same programs the small model detects and names better on source than on IR, and only the frontier gains from IR, by trading recall for a lower false-positive rate.

Model (Deployable-733)	binAcc	catch	FPR	name
Qwen-3.5-9B untuned	0.505	0.046	0.036	0.588
Qwen-3.5-9B TERSE ep3	0.537	0.372	0.299	0.533
Qwen-3.5-9B MIXED ep3	0.638	0.497	0.222	0.781
Qwen2.5-Coder-7B terse	0.527	0.428	0.374	0.975
LFM2.5-2.6B SFT r4	0.528	0.756	0.701	0.315
LFM2.5-2.6B SFT r8	0.502	0.748	0.745	0.522
LFM2.5-2.6B SFT r16	0.554	0.786	0.679	0.614
LFM2.5-2.6B SFT r32	0.519	0.642	0.604	0.684
LFM2.5-2.6B r16+exp	0.551	0.593	0.492	0.644
LFM2.5-2.6B r32+exp	0.540	0.621	0.541	0.699
LFM2.5-2.6B sec55	0.550	0.474	0.374	0.634
LFM2.5-2.6B REALSFT	0.509	0.436	0.418	0.056
LFM2.5-2.6B LLVM r128 (best IR)	0.475	0.599	0.648	0.647
gpt-5.6-luna Source	0.623	0.900	0.654	0.663
gpt-5.6-luna LLVM IR	0.706	0.775	0.363	0.661

the identical Deployable-733 subset (Section 2.3), so the two representations face the same 733 programs and neither is advantaged by sample selection. All figures are recomputed from the saved per-sample runs.

- **Source dominates IR for the small model.** The best on-device source detector (Qwen MIXED ep3, binAcc 0.638) far exceeds the best on-device IR model (LFM LLVM r128, 0.475), and every 2.6B source configuration also clears it. Meaningful source naming reaches 0.684 (LFM r32) and 0.781 (Qwen MIXED), both above IR r128’s 0.647, which IR attains only at rank 128, four times the adapter of source r32. Compiling to IR gives the small model nothing over source on the same programs.
- **IR helps the frontier, and only through calibration.** gpt-5.6-luna’s balanced accuracy rises from 0.623 on source to 0.706 on IR because IR cuts its false-positive rate from 0.654 to 0.363, while its recall falls from 0.900 to 0.775. IR trades detection sensitivity for far fewer false alarms, a shift a $>1T$ model converts into net accuracy and the 2.6B cannot.
- **The representation benefit is capacity-gated.** The abstraction that lets the frontier read past surface tokens stays out of the small model’s reach, so the 2.6B pays IR’s length and rank costs (Section 4.3) without the detection payoff. Coder-7B’s 0.975 naming is a terse-output artifact (13-token verdicts on a chance-level detector), not a naming advantage.

4.4.2 Why Synthset and Realset Diverge

The same model often looks strong on Synthset and ordinary on Realset. Qwen MIXED ep3 reaches 0.626 balanced accuracy with 0.821 naming on the synthetic set, then falls to 0.514 and 0.142 on the real set. The frontier moves the same direction, from 0.629 to 0.545. This gap is structural, and it traces to three differences between the benchmarks.

- **Distribution match.** The synthetic training traces and all of Synthset come from one generator family of 104 defect

shapes with oracle-verified labels, so a model tuned on those traces reads Synthset in-distribution and recognizes a shape close to what it trained on. Realset is real CVE code from BigVul, provably disjoint from all training data (Section 2.3), so every real program is out-of-distribution. Naming makes the effect concrete. REALSFT, the only 2.6B trained on real PrimeVul code, scores worst on Synthset naming at 0.104 and best on Realset naming at 0.288, above the frontier’s 0.278. Each model tracks the distribution it learned rather than an abstract notion of vulnerability.

- **Label determinacy inside the function.** The generator emits self-contained programs whose defect the oracle confirmed fires within the code, so the synthetic label is decidable from the function alone. Real CVE functions arrive extracted from their projects, missing the macro definitions and caller context that decide whether a vulnerable path is reachable. The real label is frequently under-determined from a single function, which pins every on-device model near chance and holds even the frontier to 0.545.
- **Discrimination difficulty.** Synthset sets a clean vulnerable program against a clean secure one, a wide separation. Realset sets a vulnerable function against its own minimal-diff fix, so the two members differ only by the patch hunk. Telling a program apart from its own near-identical repair is a far finer test than telling two unrelated programs apart, which is why real-code pair-consistency stays low even for models that score well on the synthetic split.

The two benchmarks measure different things. Synthset asks whether a model learned the shape library behind its training data, and Realset asks whether that skill survives unseen production code that has been stripped of context and separated from its own fix by a few lines. The steady drop from one to the other is the empirical form of the synthetic-to-real transfer gap, and it is why this paper reports both numbers and frames deployment around a reviewer-in-the-loop cascade rather than any single synthetic score.

5 Direct Preference Optimization (DPO)

To widen the gap between true catch and false alarm rates, I apply Direct Preference Optimization using verifier-confirmed matched pairs.

5.1 DPO Setup and Math

Each preference item consists of a program x , a preferred completion y_w , and a rejected completion y_l . Ground-truth correctness is determined by dynamic sanitizer execution. The DPO loss objective is formulated as follows.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \quad (4)$$

where π_{θ} is the active LoRA policy, π_{ref} is the frozen SFT reference model, σ is the logistic sigmoid, and $\beta = 0.1$ controls policy divergence regularization.

5.2 DPO Results

Table 12 reports DPO alignment across three evaluation sets. The Realset rows (2,995 pairs) fix the discrimination wall at each SFT start, while the measurable before-and-after of DPO runs on PrimeVul (2,481 pairs) and Synthset (957 programs), the two real and synthetic sets where the paired pre and post checkpoints were both scored.

Table 12: Direct Preference Optimization Results Across Benchmarks

Model / Stage	Benchmark	binAcc	FPR	pair
REALSFT Start	Realset	0.504	0.426	0.178
sec55 SFT Start	Realset	0.504	0.515	0.183
sec55 SFT Start	PrimeVul	0.507	0.498	0.191
DPO Post-sec55 ep2	PrimeVul	0.507	0.448	0.187
sec55 SFT Start	Synthset	0.532	0.373	N/A
DPO Post-sec55 ep2	Synthset	0.565	0.277	N/A
DPO Post-sec55 ep3	Synthset	0.537	0.275	N/A

5.3 DPO Analysis

- **DPO induces a threshold operating shift rather than class separation.** Training satisfied preference loss optimization at a 74% win rate on candidate pairs, yet on PrimeVul catch and false-positive rate fell in lockstep, 0.51 to 0.46 alongside 0.50 to 0.45. The result is a global caution shift rather than sharper discrimination.
- **Real-world minimal-diff pair consistency stays at the discrimination wall.** Both Realset SFT starts sit at chance balanced accuracy (0.504), and the PrimeVul before-and-after moves pair-consistency only from 0.191 to 0.187 with balanced accuracy flat near 0.507, so DPO leaves the wall in place.

6 Final Results Recap and Best Models

Table 13 summarizes the top-performing on-device configurations against the gpt-5.6-luna frontier baseline across all evaluation dimensions.

Table 13: Final Benchmark Comparison of Top On-Device Models vs Frontier. name/+fam is graded CWE-tree naming with a one-hop family fallback. The LLVM r128 row is scored on the deployable-733 subset.

Model Configuration	Role	Synthset (957)			Realset (2,995)			
		Acc	FPR	Name/+fam	Acc	FPR	Pair	Name/+fam
Qwen MIXED ep3 (9B)	Generalist	0.626	0.224	0.821/0.847	0.514	0.499	0.209	0.142/0.205
Coder-7B terse (7B)	Real Leader	0.512	0.365	0.972/0.976	0.510	0.510	0.226	0.209/0.312
REALSFT (2.6B)	Real Namer	0.521	0.396	0.104/0.152	0.504	0.426	0.178	0.288/0.439
LLVM SFT r128 (2.6B)	IR Specialist	0.475	0.651	0.666/0.683	N/A	N/A	N/A	N/A
gpt-5.6-luna (Frontier)	Ceiling	0.629	0.656	0.765/0.769	0.545	0.502	0.156	0.278/0.362

6.1 Combining Models and Ensemble Approach

Decomposing real-world pair-consistency (pair = split × cflip) reveals why on-device models lead on pair-consistency (0.226 vs 0.156) despite lower balanced accuracy (0.510 vs 0.545). On-device models exhibit high pair split rates (split = 0.431) with chance-level directionality (cflip = 0.524), whereas gpt-5.6-luna splits pairs rarely (split = 0.222) but with strong directionality (cflip = 0.701, ϕ = 0.564).

Furthermore, empirical analysis reveals an inter-model interference phenomenon between weakness naming and binary

detection. Mixing synthetic and real-world supervision buries real-world CWE naming (0.142 vs 0.288).

To resolve this trade-off, I evaluate voting ensembles and a two-stage detect-then-name cascade pipeline on 2,995 Realset pairs. Table 14 defines each combination strategy, and Table 15 reports their results.

Table 14: Combination Strategies Evaluated

Strategy	What it entails
Best Single Model	The strongest individual model, Coder-7B terse, run alone with no combination. The baseline any ensemble must beat.
Majority Vote (3)	Three specialists, Qwen MIXED ep3 (9B), Coder-7B terse (7B), and REALSFT (2.6B), each emit a vulnerable-or-secure verdict per function, and the function is flagged when at least two of the three agree.
Union	Flag a function vulnerable if any one of the three specialists (Qwen MIXED ep3, Coder-7B terse, REALSFT) flags it. Maximizes recall and floods false positives.
Intersection	Flag a function vulnerable only if all three specialists flag it. Maximizes precision and collapses recall.
Cascade (Detect + Name)	Two stages. A detector (Qwen MIXED ep3 for the 9B variant, or Coder-7B terse for the 7B) sets the flag, and each flagged finding is then routed to REALSFT (the 2.6B real-code model), which assigns the CWE class with a family-level fallback.
Frontier Ceiling	gpt-5.6-luna run alone, the reference line for a single hosted model.

Table 15: Real-World Ensemble and Detect-Then-Name Cascade Evaluation

Combination Strategy	Pair-Consistency	FPR	REAL Name/+fam
Best Single Model (Coder-7B)	0.226	0.510	0.209/0.312
Majority Vote (3 Models)	0.218	0.463	N/A
Union / Intersection	0.108 / 0.101	0.859 / 0.112	N/A
Cascade (Coder-7B detect + REALSFT name)	0.226	0.510	0.290/0.442
Cascade (Qwen MIXED ep3 detect + REALSFT name)	0.209	0.499	0.270/0.414
gpt-5.6-luna (Frontier Ceiling)	0.156	0.502	0.278/0.362

Majority voting (0.218) cannot beat the best single detector (0.226). However, the detect-then-name cascade achieves top performance across all axes.

1. **Generalist Screening (Qwen-3.5-9B / Coder-7B)** Serves primary zero-shot inference, matching frontier synthetic detection accuracy (0.626 vs 0.629) and real-world pair consistency (0.226 vs 0.156).
2. **Real-World CWE Verification (REALSFT 2.6B + Family Fallback)** Routes flagged real-world code candidates to the 2.6B real-code specialist with family class emission, elevating real-world naming to 0.442 (+fam), outperforming gpt-5.6-luna (0.362).

6.2 Limitations

- **Synthetic-to-Real Transfer Gap** While synthetic training transfers minimal-diff pair discrimination (on-device pair-consistency 0.226 vs frontier 0.156), absolute real-world

binary detection accuracy collapses to chance (0.50 to 0.51) for all on-device models.

- **Isolation Undecidability** Isolated real-world functions frequently lack macro definitions, global type structures, and caller contexts, capping single-function detection ceilings.
- **MoE Parameter Adaptation** Standard PEFT LoRA cannot target grouped tensor parameters in Mixture-of-Experts architectures (LFM2.5-8B-A1B), requiring full fine-tuning.
- **Model Scale Ceiling** Each fine-tuning job ran on a single 48 GB L40S, which caps trainable parameters and holds the study near a 12B ceiling. Current state-of-the-art small models such as Qwen-3.6-27B or the Gemma-4 26B and 31B variants never enter the comparison, since a 27B model in bf16 overflows a single L40S before the LLVM IR token tax is applied. Larger models carry more capacity for the long-range structure that IR encodes, so they might learn the representation far better than anything in our range. The finding that on-device models cannot learn IR therefore holds only for the scale we could afford and stays open above 12B.

6.3 Future Directions

To overcome the real-world discrimination wall and achieve production-grade automated code analysis, future work should focus on three directions. First, expanding model input beyond single functions to include repo-level inter-procedural slicing, macro expansion, and caller-callee dependency graphs to resolve isolation undecidability. Second, building automated dynamic execution harnesses for real-world CVE patches, enabling verifier-gated reinforcement learning directly on real code rather than unverified static commits. Third, scaling evaluation to 27B to 32B parameter models using multi-node distributed training, testing whether higher model capacity unlocks long-range reasoning over compiler-derived intermediate representations.

7 Conclusion

I restate and answer my three primary research questions based on empirical evidence.

RQ1 How far do 1 to 12B on-device models trail gpt-5.6-luna (high) on CWE classification, measured by balanced accuracy, naming, and pair-consistency, and how does the gap scale with model size and architecture?

On clean synthetic source code, the 9B model closes the gap entirely, matching gpt-5.6-luna on detection accuracy (0.626 vs 0.629) at one third the false positive rate (0.224 vs 0.656) and superior weakness naming (0.821 vs 0.765). On real-world source code, an absolute discrimination wall holds all on-device models at chance balanced accuracy (0.50 to 0.51) while gpt-5.6-luna reaches only 0.545. However, on-device models lead on real-world minimal-diff pair consistency (0.226 vs 0.156) and family-level naming (0.442 vs 0.362).

RQ2 Does compiler-derived LLVM IR help or hurt a small model, relative to source, for the same task?

LLVM Intermediate Representation does not help small on-device models. On-device detection accuracy remains at chance (0.509) while incurring a 6x to 9x token length tax that forces a 16,384 token training wall and excludes 23% of evaluation

programs. On the identical Deployable-733 subset the small model in fact detects and names better on source than on IR (Section 4.4). LLVM IR does benefit the frontier, though only as a calibration shift. It cuts gpt-5.6-luna’s false-positive rate from 0.654 to 0.363 but also lowers its recall of real bugs from 0.900 to 0.775, and balanced accuracy rises to 0.706 only because the false-positive reduction outweighs the recall loss. Small-model naming is learnable on IR (0.648) but its rank ceiling plateaus at rank 128.

RQ3 How much does reasoning-distillation supervised fine-tuning, and preference-based reinforcement learning on top of it, close the gap over the untuned baselines?

Off-policy reasoning-distillation SFT is the decisive lever. On the 9B model, distillation lifts synthetic detection from 0.524 up to 0.626 and real-world pair consistency from 0.099 to 0.209. In contrast, terse format-only imitation achieves only 0.141 pair consistency. Preference optimization (DPO) induces a uniform threshold caution shift along the ROC diagonal without crossing the real-world discrimination wall.

8 References

References

- [1] J. Fan, Y. Li, S. Wang, and T. N. Nguyen. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *Proc. MSR*, 2020. <https://doi.org/10.1145/3379597.3387501>
- [2] Y. Ding et al. Vulnerability detection with code language models How far are we? In *Proc. ICSE*, 2024. <https://arxiv.org/abs/2403.18624>
- [3] NIST. Juliet C/C++ test suite, Software Assurance Reference Dataset (SARD). <https://samate.nist.gov/SARD/test-suites/112>
- [4] MITRE. Common Weakness Enumeration (CWE). <https://cwe.mitre.org>
- [5] D. Gneciak and T. Szandala. Large language models versus static code analysis tools A systematic benchmark for vulnerability detection. *IEEE Access*, 2026. <https://arxiv.org/abs/2508.04448>
- [6] Y. Li et al. Revisiting pre-trained language models for vulnerability detection, 2025. In *Proc. ACM Asia CCS*, 2026. <https://arxiv.org/abs/2507.16887>
- [7] M. Al Atiq, Y. Zhou, and S. Gehrmann. Dissecting the black box Circuit-level analysis of LLM vulnerability detection, 2026. <https://arxiv.org/abs/2605.29901>
- [8] F. Müller-Uri and T. Pykälistö. Benchmarking large language models for vulnerability detection Comparing local and cloud LLMs. Technical Report, Lund University, 2026. <https://lup.lub.lu.se/student-papers/search/publication/9229367>
- [9] Z. Lin and A. Mohaisen. From large to mammoth A comparative evaluation of large language models in vulnerability detection. In *Proc. NDSS*, 2025. <https://dx.doi.org/10.14722/ndss.2025.241491>

- [10] R. Henareh. Fine-tuning small open-weight LLMs for cybersecurity. Master's thesis, KTH Royal Institute of Technology, 2026. <https://kth.diva-portal.org/smash/get/diva2:2060365/FULLTEXT01.pdf>
- [11] L. Lu et al. Evaluating and enhancing the vulnerability reasoning capabilities of large language models, 2026. <https://arxiv.org/abs/2602.06687>
- [12] A. Rondanini et al. LoRA-based parameter-efficient LLMs for continuous learning in edge-based malware detection, 2026. <https://arxiv.org/abs/2602.11655>
- [13] A. V. Kozachok and A. V. Kozachok. A review of small language models for code and domain-specific language artifact generation. *Preprints.org*, 2026. <https://www.preprints.org/manuscript/202606.1095>
- [14] J. Hu, X. Jin, Y. Zeng, Y. Liu, Y. Li, D. Du, K. Xie, and H. Zhu. QLPro Automated code vulnerability discovery via LLM and static code analysis integration, 2025. <https://arxiv.org/abs/2506.23644>
- [15] S. Jin. Large language models for vulnerability detection in static code analysis A survey, 2025. <https://ieeexplore.ieee.org/document/11082007>
- [16] Y. Xiong and T. Zhang. Sifting the noise A comparative study of LLM agents in vulnerability false positive filtering, 2026. <https://arxiv.org/abs/2601.22952>
- [17] B. Ahmed, M. Khan, and R. Islam. SECVULEVAL Benchmarking LLMs for real-world C/C++ vulnerability detection, 2026. <https://arxiv.org/abs/2505.19828>